How Full Micro, MicroMacro, and Structural A/U–E/U–E/E Actually Work

A step-by-step explanation using one network, formulas, implementation details, and comparison results

What this document is meant to clarify

All variants below simulate infection spreading on a network, but they use different amounts of information about inter-community contacts. **Full Micro** sees every edge. **Production MicroMacro** keeps detailed node-level networks inside communities but uses aggregate information between communities. **Structural A/U, E/U, and E/E** are diagnostic variants in which the information lost by aggregation is restored one component at a time.

1. Start from the same physical network

Assume two communities, A and B. Each contains an ordinary person-to-person contact network. Only a few real contacts connect the two communities. Red nodes are infected, gray nodes are susceptible. A blue outline marks a boundary/gateway node that has at least one contact with another community.

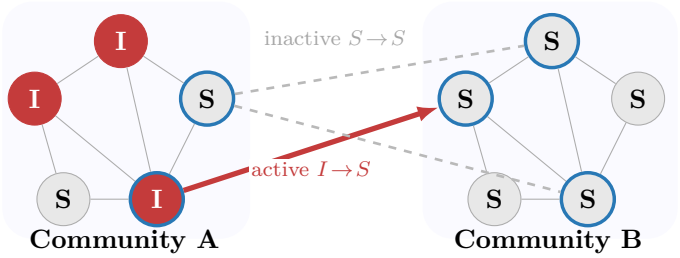


Figure 1: One physical network. There are three cross-community edges, but only one is currently active as an $I \to S$ transmission channel. Different simulators either retain or lose this distinction.

The same physical network is supplied to all simulators. The difference is not the graph itself, but which part of the graph is used when inter-community transmission is computed.

2. Full Micro: the simulator sees everything

Full Micro is a Gillespie simulation on the entire contact graph. It makes no physical distinction between an internal edge and a cross-community edge. Every active $I \to S$ edge is a CTMC transition channel:

$$a_{uv} = \beta w_{uv} \, \mathbf{1}[X_u = I, \, X_v = S].$$

The total infection hazard is the sum over all currently active channels,

$$\Theta_{\text{micro}} = \sum_{(u,v) \in E} a_{uv}.$$

Thus Full Micro effectively asks: *which particular infected people are currently connected to which particular susceptible people?*

In Fig. 1 only one of the three cross-community edges is active. If all cross-edges have weight 1 and infection rate β, the exact A→B cross-community hazard is simply

$$\lambda_{AB}^{\text{exact}} = \beta.$$

If that channel fires, the susceptible node at the actual endpoint of that edge is infected. Full Micro does not invent a target; the graph determines it.

Intuition

Full Micro = "I know the state of every person and every edge, so I know exactly which transitions are currently possible."

3. Production MicroMacro: detailed inside communities, coarse between them

MicroMacro keeps a node-level Gillespie process inside each community, but compresses inter-community information. At the macro layer a community is represented approximately by quantities such as

$$A: \quad I_A,\ S_A,\ N_A,$$

and each pair of communities has an aggregate weight W_{AB}.

The MoMLeT model writes the inter-community rate as

$$\lambda_{AB}(t) = \beta W_{AB} C_A(t) S_B(t) T,$$

where C_A is aggregate contagiousness, S_B is aggregate susceptibility, and T is a calibration factor. A useful schematic reading is

$$\lambda_{AB}^{A/U} \propto \beta W_{AB} \frac{I_A}{N_A} \frac{S_B}{N_B}.$$

For the toy network, $W_{AB} = 3$, $I_A/N_A = 2/5$, and $S_B/N_B = 1$, so an aggregate formula can give approximately

$$\lambda_{AB}^{A/U} = 3\beta \frac{2}{5} = 1.2\beta,$$

although Full Micro sees only one active cross-edge and gives β.

This is the structural approximation: the macro layer assumes infected and susceptible nodes are mixed with respect to the boundary roughly as they are in the whole community. After a macro event, production MicroMacro does not know the exact endpoint of the physical cross-edge, so it imports infection into a uniformly selected susceptible node of the destination community.

3.1. A second difference: how MicroMacro advances time

Production MicroMacro also has its own scheduler. Micro communities advance in intervals of length τ_{micro}, while the macro layer accumulates

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds$$

until it reaches a random threshold $E = -\ln U$. In the production implementation this integral is evaluated on the synchronization grid, using a midpoint-style approximation within intervals.

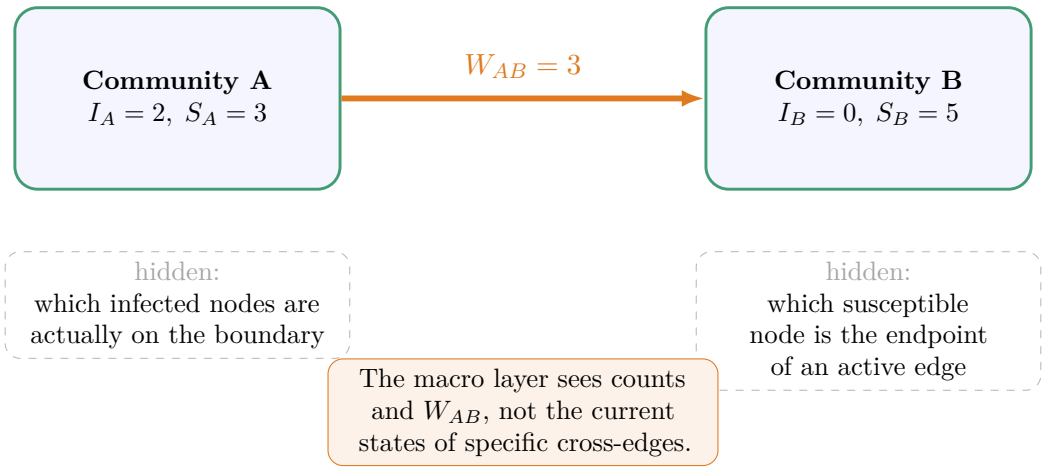


Figure 2: Macro-level view of the same network. Node-level models still exist inside communities, but production MicroMacro uses aggregate counts and W_{AB} for cross-community transmission.

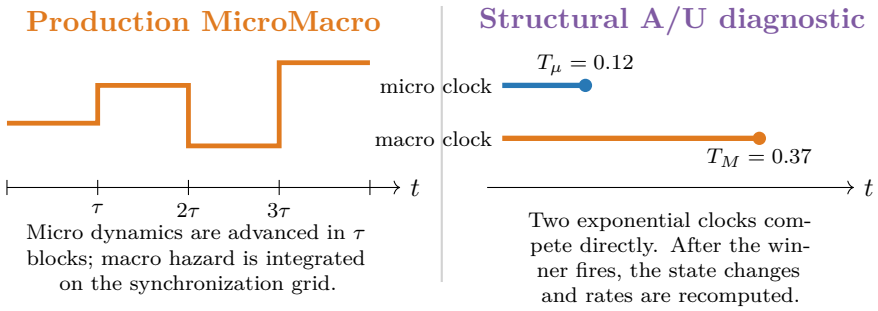


Figure 3: Scheduling difference. Production MicroMacro uses τ-synchronization and cumulative-hazard integration; A/U uses direct event competition.

The production baseline used in v4

For the numerical comparisons in v4, the primary production reference is **the same MicroMacro implementation with independent RNG streams for the micro communities**. This is not a new scientific model: aggregate hazard, uniform import placement, $\tau = 1$, and the production scheduler are unchanged. The shared-RNG variant is retained only as a buggy historical/current reference.

4. Structural A/U: the same coarse structural assumptions, but a different simulator

A/U means

$$A = \text{Aggregate hazard}, \qquad U = \text{Uniform placement}.$$

Structurally, A/U makes the same two coarse assumptions as production MicroMacro: cross-community hazard is computed from aggregate information rather than exact active cross-edges, and a macro import is placed uniformly among susceptible nodes in the destination community.

However, A/U is **not the same executable code**. The diagnostic implementation does not use the production τ-scheduler in the same form; micro and macro events compete as event-driven stochastic clocks.

Why A/U is needed

Full Micro and production MicroMacro differ for two reasons at once. First, exact boundary-edge information is replaced by aggregate hazard and uniform placement. Second, global event competition is replaced by the production τ-synchronized hazard-integration algorithm. A/U keeps the first difference but removes the second. Therefore Full Micro vs A/U primarily exposes the structural A/U approximation rather than mixing it with production scheduling mechanics.

	Production MicroMacro	**Structural A/U**
Macro information	aggregate	aggregate
Placement	uniform susceptible	uniform susceptible
Hazard idea	A/U approximation	A/U approximation
Time scheduler	production τ + hazard integral	event-driven clocks
Executable	user-facing production simulator	separate diagnostic simulator

4.1. What the A/U comparison showed

The baseline uses 100 paired networks with $L = 10$, $n = 100$, $d \approx 6$, $k = 5$, $\beta_{\text{in}} = \beta_{\text{out}} = 0.003$, SI dynamics, and $T_{\text{end}} = 4000$.

What RMST means here

RMST is the Restricted Mean Survival Time. In this experiment the event is the first arrival of infection in the final community of the chain. For replication r, let this time be $T_{\text{last}}^{(r)}$. With a finite observation horizon,

$$\text{RMST} = \frac{1}{R}\sum_{r=1}^{R} \min\left(T_{\text{last}}^{(r)}, T_{\text{end}}\right).$$

For example, three runs with terminal arrival times 2800, 3200, and "not reached by 4000" contribute 2800, 3200, and 4000, giving RMST = 3333.3. If every run finishes before the horizon, RMST is simply the ordinary mean terminal-arrival time.

Interpretation: larger RMST means slower propagation to the end of the chain. In the previous tables, $\Delta\text{RMST} = \text{RMST}_{\text{variant}} - \text{RMST}_{\text{Full Micro}}$.

v4 reporting rule

The v4 comparison does not rely on separate small RMST tables. All six implementations are compared together in Section 8 using the full empirical distributions of T_{last}. RMST is retained here only as historical context from earlier experiment reports.

A/U reached the terminal community about 221 time units earlier than Full Micro on average ($\Delta\text{RMST} \approx -220.8$; 95% CI approximately $[-315.5, -126.8]$). This already shows that aggregate hazard plus uniform placement produces systematic acceleration even without the production scheduler.

The old shared-RNG production MicroMacro was much faster still: RMST 1677.55. The gap between it and event-driven A/U was about 1038 RMST units. That gap, however, must **not** be interpreted as a pure τ-scheduler error. After correcting the shared-RNG architecture, production RMST became 2729.43 and differed from A/U by only about 13.5 RMST units; the corresponding confidence interval included zero. Reducing τ from 1 to 0.25 changed RMST by only about 31.15.

What "Production + independent RNG streams" means

These are not two scientific models. Both variants use the same production MicroMacro: the same local MicroModels, aggregate macro hazard, uniform import placement, $\tau = 1$, midpoint/cumulative-hazard scheduler, and normal repeated imports. The only change is the source of random numbers used by each micro community.

In the current/shared-RNG implementation, MicroModels are created without private RNG objects, so multiple communities use the same module-level Python RNG state. A shared pseudorandom generator alone is not necessarily wrong. The problem appears in combination with **horizon-safe rollback**. When `simulate_until(t_end)` samples a Gillespie event beyond the current horizon, it restores the RNG to the state before the draw. With a private RNG this is local. With a shared global RNG, rolling back one community also rewinds the random stream that a later community will see.

Why this creates artificial dependence

Suppose the global RNG state is R before community A is simulated. A draws a future event, finds it beyond the horizon, and restores the RNG back to R. Community B is simulated next and starts from that same restored state. It can therefore reuse the same random-number sequence. B's stochastic evolution becomes dependent on whether A attempted and rolled back a draw. This is cross-community RNG coupling and execution-order dependence, not a physical interaction between communities.

With independent streams, each micro community has its own generator,

$$\mathrm{RNG}_1, \mathrm{RNG}_2, \ldots, \mathrm{RNG}_L.$$

Rollback in community i modifies only RNG_i. Midpoint clones also copy and restore the corresponding per-community RNG state. Macro budget and macro pair selection remain part of the original production machinery; the scientific aggregate A/U semantics and the τ-scheduler are unchanged.

A dedicated unit diagnostic confirmed the effect: two identical isolated micro processes had correlation of final infected counts about 0.531 with the shared-global RNG and about -0.002 with independent streams. More importantly, the probability of infecting the target was about 0.394 for the first community but only 0.155 for the second under shared RNG; after splitting the streams the values were about 0.394 and 0.390. Thus shared RNG plus rollback changed even the marginal distribution of the second community.

Implementation	**Micro scheduling/state**	**Cross-community model**	**RNG architecture**
Full Micro	one global Gillespie process on the full graph	exact active edges, exact endpoint	one RNG for one global process
Current Production MM	separate MicroModels + τ synchronization	aggregate hazard + uniform import	one shared global RNG across micro communities
Production + independent RNG	same production scheduler	same aggregate/uniform model	separate RNG for each micro community
Structural A/U	one diagnostic global state; event-driven race	aggregate hazard + uniform placement	diagnostic streams by stochastic role
E/U, E/E	same diagnostic event-driven framework	exact hazard; E/U uniform, E/E exact placement	diagnostic streams; not production per-community architecture

5. E/U: correct only the hazard

E/U keeps uniform placement but restores exact information about real cross-community edges. For pair A→B,

$$\lambda_{AB}^{E/U} = \beta \sum_{(u,v)\in E_{AB}} w_{uv}\mathbf{1}[X_u = I,\, X_v = S].$$

In the toy network only one cross-edge is active, so $\lambda_{AB}^{E/U} = \beta$ rather than the aggregate approximation 1.2β.

However, after the macro event occurs, E/U still performs "U": it chooses any susceptible node in B rather than necessarily the endpoint of the active cross-edge. Thus A/U→E/U primarily isolates the error introduced by aggregate hazard. Both use the same diagnostic event-driven scheduler and both retain uniform placement; only the hazard provider changes.

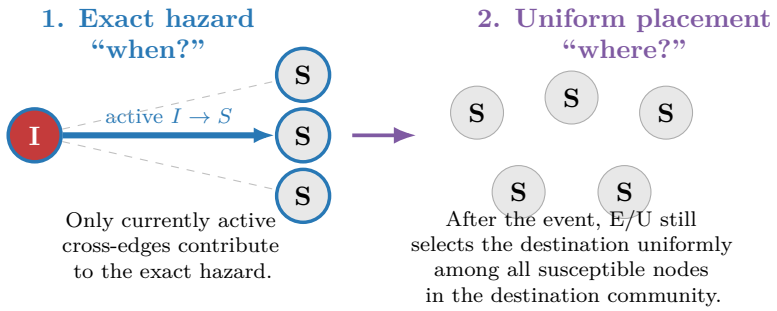


Figure 4: E/U fixes only the hazard: exact active edges determine the event intensity, but the post-event target is still uniform.

On the baseline, replacing aggregate hazard with exact active-edge hazard increased RMST by about 59.6 relative to A/U, moving the diagnostic simulator toward Full Micro. A significant gap remained, about −161.2 RMST relative to Full Micro (95% CI $[-266.8, -48.8]$). The remaining error is explained by placement: E/U knows *when* a cross-community event should occur but can still put the import into the wrong susceptible node.

6. E/E: correct both hazard and placement

E/E means

$$E = \text{Exact hazard}, \qquad E = \text{Exact placement}.$$

The hazard is the same exact active-edge sum as in E/U. The difference is that when a cross-edge (u, v) is selected, the infection is placed at its actual endpoint v. Thus E/E restores both the event intensity and the allowed destination endpoint.

The transition E/U→E/E isolates the uniform-placement error: scheduler and exact hazard remain unchanged, while only target selection changes.

On the baseline, E/E was almost indistinguishable from Full Micro. The RMST difference was only about +7.5 with a 95% CI of approximately $[-93.6, +120.3]$. E/U→E/E increased RMST by about 168.8, moving propagation later and much closer to Full Micro. This indicates that, once hazard is correct, placement is a major remaining source of structural error.

7. What is the actual difference between E/E and Full Micro?

After checking the implementations, this can be stated more strongly: **in Experiment 01, E/E and Full Micro are configured to implement the same CTMC generator**. They differ in

code organization and random-number routing, not in the infection rules.

Full Micro receives the entire `full_graph` and passes it to one `MicroEngine`. Internal and cross-community edges are simply edges of the same graph. Each active $u(I) - v(S)$ edge creates a channel

$$a_{uv} = \beta w_{uv},$$

and each infected node has recovery rate γ. All infection and recovery channels compete in one Gillespie race.

E/E also uses one global state for all nodes. Importantly, it does **not** contain a separate "micro A simulator" and "micro B simulator". The code only indexes edges into two groups: internal edges and cross-community edges. An active internal edge has rate $\beta_{\text{in}} w$ and an active cross-edge has rate $\beta_{\text{out}} w$.

The Experiment 01 runner explicitly requires

$$\beta_{\text{in}} = \beta_{\text{out}},$$

because Full Micro uses one infection rate for all graph edges. Recovery rules are the same as well.

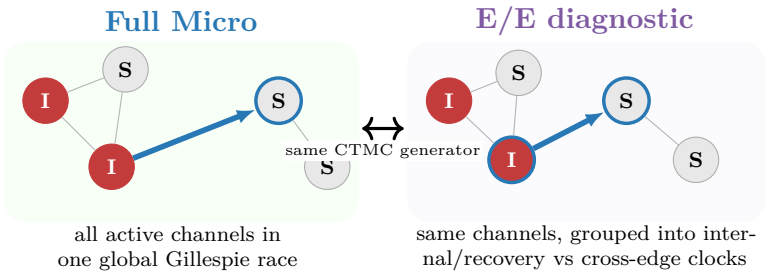


Figure 5: E/E and Full Micro organize the channels differently in code, but in Experiment 01 they implement the same CTMC generator.

Let

$$\Lambda_\mu = \Lambda_{\text{internal}} + \Lambda_{\text{recovery}}, \qquad \Lambda_M = \Lambda_{\text{cross}}.$$

Full Micro samples one waiting time

$$\Delta t \sim \text{Exp}(\Lambda_\mu + \Lambda_M).$$

E/E samples two independent times

$$T_\mu \sim \text{Exp}(\Lambda_\mu), \qquad T_M \sim \text{Exp}(\Lambda_M)$$

and takes the smaller. By the competing-exponentials identity,

$$\min(T_\mu, T_M) \sim \text{Exp}(\Lambda_\mu + \Lambda_M),$$

and the probability that a group wins is proportional to its total rate. Thus the two scheduling procedures are mathematically equivalent.

For cross-community events, E/E also preserves the exact probability of selecting each active cross-edge. It first selects a community pair proportionally to the sum of rates of its active cross-edges, then selects the target proportionally to the total active-edge weight entering that target, and finally selects the source proportionally to the relevant edge weight. The resulting probability of choosing a concrete (u, v) edge is the same as in Full Micro.

No "if the rules match" qualifier is needed for Experiment 01

The implementation itself enforces the equivalence conditions: same full graph and initial node, $\beta_{\text{in}} = \beta_{\text{out}}$, same γ, same SIR event rules, exact active cross-edge hazard, and exact edge-conditioned placement. Therefore E/E and Full Micro should have the same statistical distribution even though individual trajectories differ because RNG streams are organized differently.

The oracle gate agrees with this: in 100 paired replications, Full Micro RMST was 2936.7 and E/E RMST was 2944.3; the difference was +7.5 with 95% CI $[-93.6, +120.3]$, which includes zero.

8. One matched comparison of all simulator variants

For v4, all available variants are combined into one matched comparison on the same 100 paired networks: $L = 10$, $n = 100$, $d \approx 6$, $k = 5$, $\beta_{\text{in}} = \beta_{\text{out}} = 0.003$, SI dynamics, and $T_{\text{end}} = 4000$. The archived production and structural per-replication files use the same replication IDs; Full Micro values match exactly and EventDriven A/U matches A/U. Therefore the rows below refer to the same network realizations rather than different experimental samples.

All six variants reached the terminal community in all 100/100 runs before $T_{\text{end}} = 4000$, so there is no censoring in this distribution-first comparison.

8.1. Compare distributions, not only one average

For simulator m, consider the complete terminal-arrival distribution

$$F_m(t) = P(T_{\text{last}} \leq t).$$

Median and IQR are retained as descriptive summaries, but the primary pairwise quantities are distances between distributions:

- **Wasserstein-1 distance** W_1: the average amount of time-axis displacement needed to transform one empirical arrival-time distribution into another. It has the same time units as T_{last}. Zero means identical empirical distributions.
- **Kolmogorov–Smirnov distance** D_{KS}: the maximum vertical gap between two empirical CDFs. It ranges from 0 to 1; zero means identical ECDFs.

Distances are reported both relative to Full Micro and relative to **Production + independent RNG**, because corrected Production is now the main MicroMacro reference. P-values are deliberately not the central result here; for the paired-network design, distributional effect sizes are more informative.

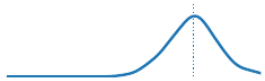
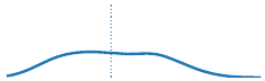
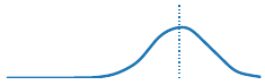
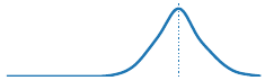
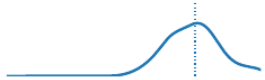
Simulator	Success	Median [IQR]	W_1 vs Full	D_{KS} vs Full	W_1 vs indep. MM	D_{KS} vs indep. MM	Density
Full Micro	100/100	2933 [2680; 3146]	0.0	0.00	207.3	0.24	
Production current (shared RNG; buggy ref.)	100/100	1649 [1076; 2302]	1259.2	0.77	1051.9	0.64	
Production + independent RNG	100/100	2715 [2436; 2994]	207.3	0.24	0.0	0.00	
Structural A/U	100/100	2709 [2516; 2890]	220.8	0.31	61.9	0.12	
Structural E/U	100/100	2796 [2433; 3063]	163.6	0.20	50.4	0.09	
Structural E/E	100/100	2963 [2624; 3202]	**40.4**	**0.07**	214.8	0.23	

Figure 6: Distribution-first comparison of all six implementations on the same 100 paired networks. Mini-density plots use a common time axis from 0 to 4000 and a common vertical scale; the dotted line marks the median. Distances from Full Micro to corrected MicroMacro are shown as symmetric reference values, not as an "error" of Full Micro.

8.2. How to read the v4 comparison

Current shared-RNG Production is an obvious outlier: $W_1 = 1259.2$ from Full Micro and 1051.9 from corrected Production. It should no longer be used as the scientific MicroMacro baseline; it remains useful as evidence of the implementation bug.

Production + independent RNG brings the production implementation into the same time regime as diagnostic A/U and E/U. The distance from corrected Production to A/U is only $W_1 = 61.9$, $D_{KS} = 0.12$; to E/U it is $W_1 = 50.4$, $D_{KS} = 0.09$. This does not mean corrected Production *is* E/U: it still uses aggregate hazard, uniform placement, and the production τ-scheduler. It only means their marginal arrival-time distributions are close in this baseline.

E/E is the closest to Full Micro: $W_1 = 40.4$ and $D_{KS} = 0.07$. This is a stronger statement than "the averages are close": the empirical terminal-arrival distributions themselves nearly coincide.

A/U and E/U lie between corrected Production and Full Micro. A/U→E/U reduces W_1 to Full Micro from 220.8 to 163.6; E/U→E/E reduces it further to 40.4. Thus the ablation ladder can be evaluated as progressive convergence of whole distributions, not merely a shift in a mean.

8.3. Structural ablation ladder

Two distinct mechanisms can make production MicroMacro differ from Full Micro:

Structural:	exact boundary edges → aggregate community hazard.
Scheduler:	global event competition → τ-synchronized hazard integration.

A/U removes the production scheduler while retaining aggregate/uniform semantics. A/U→E/U restores exact hazard. E/U→E/E restores exact placement.

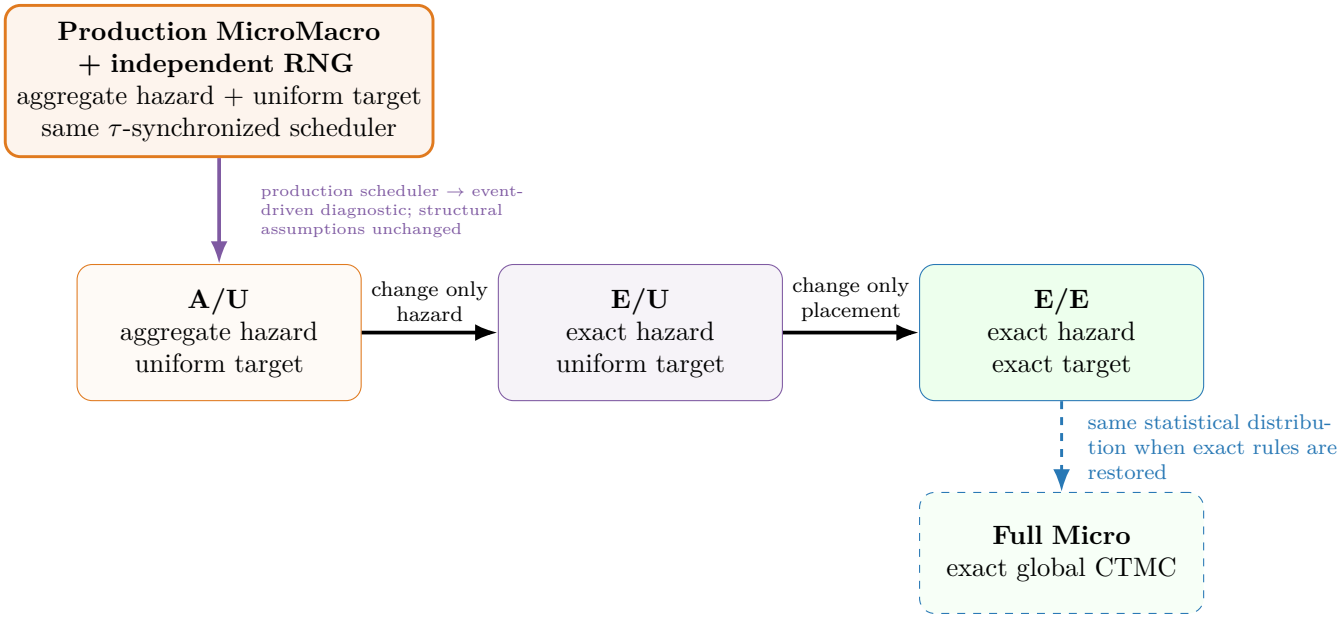


Figure 7: What each step isolates in v4. The production baseline already uses independent RNG streams.

9. What should count as the main result for the paper?

If the goal is to make claims about the user-facing MicroMacro method, the primary experiments should use the production `MicroMacroSimulator` with **independent RNG streams** and then

modify only the minimum component needed for the particular ablation. The shared-RNG variant is no longer a valid scientific baseline.

Experiment	Cleanest implementation strategy
A/U baseline	production MicroMacro + independent RNG streams; change nothing else
E/U	same corrected Production, replacing only the macro hazard provider with exact active-edge hazard
E/E	same corrected Production with exact hazard, replacing only the target selector with the exact edge endpoint
Gateway near/random/far	do not modify corrected Production; change topology only
$d \times k$ **grid**	do not modify corrected Production; change network parameters only

The separate `HazardPlacementSimulator` remains very useful as a secondary mechanistic diagnostic because it removes production scheduler artifacts and cleanly decomposes structural error. But it should be presented as mechanism validation rather than the only evidence for what a user of corrected production MicroMacro will observe.

10. Shortest possible summary

- **Full Micro**: sees every node and every active edge. Exact hazard, exact target.
- **Production MicroMacro (primary v4 reference)**: original architecture with independent RNG streams for micro communities; between communities it uses aggregate I, S, W, aggregate hazard, uniform target, and the production τ-scheduler.
- **Shared-RNG Production**: same intended architecture but with a confirmed RNG-coupling bug; keep only as a historical/bug reference.
- **Structural A/U**: same aggregate/uniform structural assumptions in a separate event-driven diagnostic simulator; it removes the production τ-scheduler while retaining structural A/U approximation.
- **E/U**: same diagnostic scheduler and uniform placement, but aggregate hazard is replaced by exact cross-edge hazard; A/U→E/U isolates hazard-information loss.
- **E/E**: same diagnostic scheduler and exact hazard, but uniform placement is replaced by the exact endpoint; E/U→E/E isolates placement error. In Experiment 01 it implements the same CTMC generator as Full Micro.

Source note

The description of production MicroMacro, its two layers, macro-rate, and cumulative-hazard synchronization is based on *Towards complexity reduction of large-scale epidemic simulation in two-scale networks* (MoMLeT-2025). The Full Micro description is based on *Simulating epidemic peak dynamics on complex networks using efficient Gillespie algorithms.* The distinction between Structural A/U–E/U–E/E is based on the audited experiment source code. Earlier RMST values come from Experiment 01 (hazard/placement factorial ablation, 100 paired replications) and Experiment 07 (production implementation decomposition). The v4 distribution-first comparison reuses the archived per-replication results; no new simulation run was needed.